URBAN AI DASHBOARD

Product Requirements Document — Version 3.0

Geo-MLLM Academic Research Platform

Full Pipeline: Data Collection → GNN → Fusion → Custom MLLM → Digital Twin

Status	Version	Supersedes	Target
Final Draft	1.0	v1.0 + v2.0	MVP + All 5 Gaps Resolved

What changed from v2.0

5 critical gaps identified in the gap analysis are now fully resolved:

Gap 1: GNN model support added — replaces MLP-only approach
Gap 2: Custom small MLLM Builder interface added as MVP feature
Gap 3: Digital Twin NL query promoted from post-MVP to MVP
Gap 4: Text modality added as independent checkbox alongside POI / Image / Graph
Gap 5: Evaluation panel added with Accuracy, F1, and Spatial Accuracy metrics

1. Introduction & Purpose

1.1 Purpose

This PRD (v3.0) is the definitive requirements document for the Urban AI Dashboard — a web-based platform serving as the interactive front-end for the MLLM-Geo-AI research project. It fully covers the 12-phase academic pipeline defined in the student project document 'A Lightweight Multi-Modal GeoAI Model for Urban Digital Twin Using OSM POI, Road Networks, and Satellite Imagery'.

Version 3.0 supersedes v2.0 by filling all five gaps identified during qualification review. The platform now supports the complete research pipeline: multi-modal data collection, graph construction, GNN-based spatial reasoning, multi-modal fusion, custom small MLLM building, model evaluation, and Digital Twin deployment with Arabic/English geo-queries.

1.2 Project Objective (Student Document Alignment)

The student project aims to build a Custom Geo-MLLM — a lightweight multi-modal language model that understands geography. The dashboard is the interactive surface where every phase of that research is driven, monitored, and validated.

Student doc phase	What the phase produces	Dashboard must support
Phase 2 — Data collection	POI, roads, satellite loaded	Load Area workflow + layer preview
Phase 3 — Data cleaning	Unified WGS84 dataset	Backend; no UI needed
Phase 4 — Graph construction	G=(V,E) road graph	Graph topology layer on map
Phase 6 — Fusion	X = POI+Image+Graph+Text vector	Fusion method selector + 4 modality checkboxes
Phase 7 — Base models	GNN + CNN + Text Encoder	Model selector (GNN vs MLP)
Phase 8 — Custom MLLM	Small LLM with geo-embeddings	MLLM Builder interface (MVP)
Phase 9 — Training	Trained Geo-MLLM weights	Training Lab with live loss chart
Phase 10 — Evaluation	Accuracy, F1, Spatial Accuracy	Evaluation Panel with metric display
Phase 11 — Digital Twin	Interactive geo-query map	NL query (MVP) + map layers
Phase 12 — Geo-MLLM output	Downloadable model	Model export + Q&A demo panel

1.3 Target Users

Persona	Goals	Pain Points
Leila — Urban Planner	Classify zones, export maps, run NL geo-queries	Manual digitisation; wants Arabic query support
Omar — GIS Analyst	Compare modalities, overlay graph topology, download GeoJSON	Needs GIS-ready formats + ablation tools
Dr. Nour — Researcher	Build and evaluate custom Geo-MLLM, run ablation studies	No visual interface for MLLM training pipeline
Karim — Developer	Monitor pipeline, test endpoints, use mock mode	No integrated test UI or evaluation dashboard

2. User Stories

ID	As a…	I want to…	So that…
US-01	Urban planner	draw a bbox or search by city name	I define the study area without code
US-02	GIS analyst	choose grid size 200m/500m/1km	I control classification granularity
US-03	Any user	see cell estimate before loading	I know job scope before starting
US-04	Any user	check 4 modalities independently (POI / Image / Graph / Text)	I run ablation studies matching the student doc
US-05	Any user	select GNN or MLP as the classifier model	I use the correct model for my research phase
US-06	GIS analyst	see the road graph topology (nodes + edges) on the map	I understand the spatial structure feeding the GNN
US-07	Any user	click Classify and see progressive results with confidence opacity	I understand certainty at a glance
US-08	Any user	click a cell to see scores, POI, satellite, road density	I understand evidence behind each result
US-09	Researcher	open MLLM Builder and select a base LLM	I start building my custom Geo-MLLM
US-10	Researcher	run the fusion-to-LLM pipeline and monitor training	I build a geo-aware language model
US-11	Researcher	see Accuracy, F1, and Spatial Accuracy after classification	I evaluate model quality against the academic standard

ID	As a…	I want to…	So that…
US-12	Researcher	upload ground-truth labels for scoring	I get real metrics, not just visual inspection
US-13	Urban planner	type 'show commercial areas near residential zones' in Arabic or English	I query the Digital Twin naturally
US-14	Any user	download the trained Geo-MLLM weights	I can deploy or share the model
US-15	Developer	toggle Mock Mode for full pipeline demo	I build and demo without infrastructure
US-16	Any user	use keyboard shortcuts (D/C/E/Esc)	I work at GIS-analyst speed

3. Functional Requirements

3.1 Area Selection & Data Loading

- FR-01: MapLibre GL map with bounding box drawing tool activated by key D or sidebar button.
- FR-02: Geocoding search box — place name to bbox via Nominatim API.
- FR-03: On area confirmation, call POST /api/v1/load-area with { bbox, place_name, grid_size, modalities: [poi, image, graph, text] }.
- FR-04: Real-time loading progress overlay with step labels: downloading_poi, downloading_roads, downloading_satellite, building_graph.
- FR-05: Client-side cell count estimate displayed before loading (formula: cells = ceil(width_m/grid_size) x ceil(height_m/grid_size)). Warn amber >300, block red >500.

3.2 Grid & Graph Configuration

- FR-06: Grid size radio: 200m / 500m (default) / 1km.
- FR-07: After load, call GET /api/v1/grid/{grid_id}/preview and render empty cell outlines on map.
- FR-08: Graph topology toggle — when enabled, render road nodes (dots) and edges (lines) on map as a separate layer. Data comes from GET /api/v1/grid/{grid_id}/graph-topology.

GAP GAP 4 RESOLVED — NOW RESOLVED

Text is now a fourth independent modality checkbox, separate from POI. This allows ablation studies matching Phase 6D of the student doc (POI-only, Image-only, Graph-only, Text-only, All).

3.3 Modality Selection (4 Independent Checkboxes)

- FR-09: Four independent modality checkboxes: POI (MLLM embedding), Image (ResNet CNN), Graph (OSMnx features), Text (BERT/MiniLM encoder).
- FR-10: At least one modality must be selected before classification is enabled.
- FR-11: Ablation presets dropdown: POI only / Image only / Graph only / Text only / POI+Image / POI+Graph / All modalities.
- FR-12: Selected modalities passed to POST /api/v1/classify as the modalities array.

GAP GAP 1 RESOLVED — NOW RESOLVED

GNN model selector added. The dashboard now supports both GNN (PyTorch Geometric, for spatial intelligence) and MLP as classifier options. GNN requires graph topology to be loaded first.

3.4 Model Selector (GNN vs MLP) — New in v3.0

- FR-13: Model type selector in sidebar: MLP (default, fast) or GNN (spatial, requires graph modality).
- FR-14: When GNN is selected, the Graph modality checkbox is automatically enabled and locked. Show tooltip: 'GNN requires road graph data'.
- FR-15: Model selection passed as model_type field in POST /api/v1/classify.
- FR-16: Graph topology layer auto-enables on map when GNN is selected, showing road nodes and edges so the researcher can see what the GNN is reasoning over.

3.5 Classification Execution

- FR-17: Classify button triggers POST /api/v1/classify with { grid_id, modalities, model_type, fusion_method, model_version }.
- FR-18: Real-time WebSocket progress. Fallback to polling GET /classify-status/{job_id}. Auto-reconnect with exponential backoff (1s, 2s, 4s, 8s max).
- FR-19: Progressive cell reveal — cells animate onto map as each batch completes. Opacity = 0.35 + (confidence * 0.60).
- FR-20: Cancel button visible during classification. Calls DELETE /api/v1/jobs/{job_id}.

3.6 Result Visualisation

- FR-21: Colour-coded grid: Residential #FFD966, Commercial #E63946, Industrial #9B5DE5. Opacity encodes confidence.
- FR-22: Map legend with class-to-colour mapping.
- FR-23: Cell click opens detail panel: dominant class, 3-class confidence bars, top-5 POI categories, satellite thumbnail, road density, node count.
- FR-24: Cell comparison pins — pin up to 3 cells for side-by-side comparison.
- FR-25: Distribution histogram in status bar — live Res/Com/Ind % bars updating per batch.

GAP GAP 5 RESOLVED — NOW RESOLVED

Evaluation Panel added as MVP. After classification, users can view Accuracy, F1-score (per class), and Spatial Accuracy. Optional ground-truth CSV upload enables real metric computation.

3.7 Evaluation Panel — New in v3.0

- FR-26: After classification completes, an Evaluation tab appears in the right panel.
- FR-27: Without ground truth, the panel shows: dominant class distribution, average confidence per class, and confidence histogram.
- FR-28: Ground truth upload — CSV or GeoJSON with cell_id and true_class columns. Calls POST /api/v1/evaluate with { job_id, ground_truth_file }.
- FR-29: With ground truth, show: Overall Accuracy (%), F1-score per class (Residential, Commercial, Industrial), Macro F1, Weighted F1, Spatial Accuracy (% of cells where dominant class matches and is spatially contiguous with neighbours of same class).
- FR-30: Confusion matrix displayed as a 3x3 colour-coded table.
- FR-31: Metrics update in the ablation comparison panel when multiple runs are compared.

3.8 Layer Controls

- FR-32: Layer toggles: Classification result, POI heatmap, Road network (edges), Graph topology (nodes + edges, for GNN), Satellite base map.
- FR-33: Layer state persists in URL query string for sharing.

GAP GAP 2 RESOLVED — NOW RESOLVED

MLLM Builder is now an MVP feature, not post-MVP. It provides the interface for Phase 8 of the student doc: selecting a base LLM, running the fusion-to-LLM pipeline, and exporting the resulting Geo-MLLM.

3.9 MLLM Builder — New in v3.0 (Phase 8 of Student Doc)

The MLLM Builder is a dedicated page (route: /mllm-builder) that enables the researcher to build a custom Geo-MLLM by feeding multi-modal embeddings into a small base language model.

3.9.1 Base Model Selection

- FR-34: Base LLM selector dropdown: LLaMA-3.2-1B, DistilGPT-2, BERT-base-multilingual, TinyLlama-1.1B. Default: DistilGPT-2 (fastest for academic GPU).
- FR-35: Embedding source selector: use embeddings from last classify job OR upload a pre-computed embedding file (.npy or .pkl).

- FR-36: Fusion method selector for MLLM input: Concatenation, Weighted, Attention (default: Concatenation).

3.9.2 MLLM Training Configuration

- FR-37: Hyperparameter form: learning rate (default 2e-5), epochs (default 10), batch size (default 16), LoRA rank (default 8 — enables LoRA fine-tuning), quantization toggle (4-bit / 8-bit / none).
- FR-38: Fine-tuning method selector: Full fine-tune, LoRA, QLoRA. Default: LoRA (memory-efficient for RTX 2070).
- FR-39: Training task selector: Classification (land use), Contrastive (embedding alignment), Both.

3.9.3 MLLM Training Execution

- FR-40: Start MLLM Training button triggers POST /api/v1/mllm/train with full config.
- FR-41: Live training chart (Recharts): loss curve, accuracy curve, both updating per epoch via WebSocket.
- FR-42: Training log panel showing real-time epoch output.
- FR-43: Cancel MLLM training: DELETE /api/v1/jobs/{job_id}.

3.9.4 MLLM Export & Demo

- FR-44: After training, Download Model button: GET /api/v1/mllm/export/{job_id}?format=pytorch|gguf|onnx.
- FR-45: Model card auto-generated and downloadable as PDF: architecture, dataset, metrics, fusion config, training hyperparameters.

GAP GAP 3 RESOLVED — NOW RESOLVED

Digital Twin NL Query is now MVP. The dashboard supports Arabic and English geo-queries against the classified grid, fulfilling Phase 11 of the student doc. Query interface includes multilingual input, result highlighting on map, and spatial explanation.

3.10 Digital Twin — NL Geo-Query (MVP) — Phase 11

The Digital Twin page (route: /digital-twin) represents the final output of the research project — an interactive geo-intelligent interface powered by the trained Geo-MLLM.

- FR-46: Natural language query input supporting Arabic and English (RTL/LTR auto-detect).
- FR-47: Example queries shown as clickable chips: 'Show residential areas with high road density' / 'اين المناطق التجارية في القاهرة؟' / 'Find industrial zones near roads'.
- FR-48: Query calls POST /api/v1/query with { question, grid_id, language: auto|ar|en }.
- FR-49: Response highlights matching cells on map with a distinct colour (#00B4D8 cyan for query results).

- FR-50: Spatial explanation shown in panel: natural language description of why those cells matched (e.g. 'These 12 cells are classified Residential with road density > 4.0 km/km²').
- FR-51: Query history panel — last 10 queries with one-click replay.
- FR-52: Geo-MLLM model selector: use trained model from MLLM Builder OR the default MiniLM-based pipeline.

3.11 Training Lab (Classification — Phase 9)

Separate from the MLLM Builder, the Training Lab handles supervised classification training (MLP or GNN fine-tuning on labelled ground truth).

- FR-53: Upload labelled grid (GeoJSON or CSV with cell_id + true_class).
- FR-54: Training config: learning rate, epochs, fusion method, model type (GNN / MLP), loss function (CrossEntropy / Contrastive / Both).
- FR-55: Start training calls POST /api/v1/train. Live loss + accuracy chart via WebSocket.
- FR-56: After training, apply new weights to classify the current grid and see updated results immediately.

3.12 Ablation Comparison Mode

- FR-57: Ablation mode lets user select 2-4 modality combinations, runs classification for each, and shows results side-by-side.
- FR-58: Comparison view: grid maps side-by-side with metric cards (Accuracy, Macro F1 if ground truth present).
- FR-59: Delta map option: show cells where classification changed between runs, coloured by change type.

3.13 Export

- FR-60: Export classification: GeoJSON / Shapefile / CSV — GET /api/v1/export/{job_id}?format=x.
- FR-61: Export map as PNG via canvas capture.
- FR-62: Export evaluation metrics as CSV: GET /api/v1/evaluate/{job_id}/export.
- FR-63: Export MLLM model weights — pytorch (.pt), GGUF (for llama.cpp), ONNX.

3.14 Mock Data Mode

- FR-64: VITE_MOCK_MODE=true activates MSW — all endpoints intercepted. Visible amber 'MOCK MODE' banner.
- FR-65: Mock dataset: Cairo 144 cells, realistic confidences (0.51–0.97), 4 modalities, GNN + MLP results both available.
- FR-66: Mock MLLM Builder: simulates 10-epoch training with live chart over 15 seconds.

- FR-67: Mock NL query: returns pre-defined cell sets for 5 demo questions in Arabic and English.
- FR-68: Mock evaluation: returns realistic Accuracy=0.81, F1=0.78, SpatialAccuracy=0.74 for demo purposes.

3.15 Keyboard Shortcuts

Key	Action
D	Activate bbox draw mode
Esc	Cancel draw / close panel
C	Trigger Classify (when area loaded)
E	Open export dialog
L	Toggle classification layer
G	Toggle graph topology layer
Q	Focus NL query input (Digital Twin page)
?	Show keyboard shortcut reference

4. Non-Functional Requirements

ID	Category	Requirement	Metric
NFR-01	Performance	Initial map load	< 3 seconds
NFR-02	Performance	Classification result (100 cells, GPU)	< 30 seconds
NFR-03	Performance	Map rendering up to 500 cells	No visible lag (vector tiles)
NFR-04	Performance	MLLM training feedback latency	Epoch chart updates within 1s of WebSocket event
NFR-05	Usability	Desktop responsive	Minimum 1280x720, target 1920x1080
NFR-06	Usability	Arabic RTL support	NL query input and results render correctly RTL
NFR-07	Reliability	WebSocket failure	Auto-reconnect + polling fallback within 3s

ID	Category	Requirement	Metric
NFR-08	Reliability	MLLM training crash recovery	Save checkpoint every 2 epochs; resume from last checkpoint
NFR-09	Security	API auth	API key / JWT on all endpoints for public deployment
NFR-10	Mock Coverage	All MVP endpoints mocked	100% of MVP routes have MSW handlers
NFR-11	Evaluation	Metric precision	F1 computed with sklearn metrics standard; Spatial Accuracy uses 8-neighbor contiguity

5. API Contract (v3.0 — Complete)

All endpoints from v2.0 are retained. New endpoints added for GNN, evaluation, MLLM Builder, and Digital Twin query. All v2.0 corrections remain in force.

5.1 Data Loading

POST /api/v1/load-area

```
{ "bbox": [31.10, 29.90, 31.30, 30.10],
  "place_name": "Cairo, Egypt",          // optional
  "grid_size": 500,                       // 200 | 500 | 1000
  "modalities": ["poi","image","graph","text"]  // all 4 now valid
}
```

GET /api/v1/grid/{grid_id}/preview

GeoJSON of empty grid cells for overlay before classification.

GET /api/v1/grid/{grid_id}/graph-topology (NEW)

Returns GeoJSON with road nodes (Point features) and edges (LineString features) for the graph topology layer. Used by the GNN model and visualised on map.

```
Response: { "type":"FeatureCollection", "features":[
  { "type":"Feature", "geometry":{"type":"Point","coordinates":[...]},
    "properties":{ "node_id":123, "degree":4, "centrality":0.12 } },
  { "type":"Feature",
"geometry":{"type":"LineString","coordinates":[[...],[...]]},
```

```
    "properties":{ "edge_id":"123-456", "length_m":312, "highway":"primary"
} }
]}
```

5.2 Classification

POST /api/v1/classify

```
{ "grid_id": "grid_abc123",
  "modalities": ["poi","image","graph","text"],  // any subset of 4
  "model_type": "gnn",                           // NEW: "gnn" | "mlp"
(default "mlp")
  "fusion_method": "concat",                     // concat | weighted |
attention
  "model_version": "v1.0"                        // optional
}
```

GET /api/v1/classification-result/{job_id}

GeoJSON FeatureCollection. Each feature properties:

```
{ "cell_id": 42,
  "dominant_class": "Residential",
  "confidences": { "Residential":0.85, "Commercial":0.10, "Industrial":0.05
},
  "poi_top_categories": ["cafe","school","bank"],
  "road_density_km_per_km2": 4.2,
  "node_count": 18,
  "satellite_thumbnail_url": "/api/v1/thumbnails/grid_abc123/cell_42.jpg",
  "graph_embedding_norm": 0.72,   // NEW: L2 norm of graph feature vector
  "text_embedding_norm": 0.61     // NEW: L2 norm of text embedding
}
```

5.3 Evaluation (NEW)

POST /api/v1/evaluate

Compute evaluation metrics by comparing classification result against uploaded ground truth.

```
Request: multipart/form-data
  job_id: "class_job_xyz789"
  ground_truth: <file: CSV or GeoJSON with cell_id + true_class>
```

```
Response: {
  "overall_accuracy": 0.81,
  "macro_f1": 0.78,
  "weighted_f1": 0.80,
  "spatial_accuracy": 0.74,
  "per_class_f1": {
    "Residential": 0.85, "Commercial": 0.72, "Industrial": 0.68 },
```

```
  "confusion_matrix": [[82,8,2],[5,44,6],[3,4,26]],
  "class_labels": ["Residential","Commercial","Industrial"]
}
```

GET /api/v1/evaluate/{job_id}/export

Download evaluation metrics as CSV.

5.4 MLLM Builder (NEW)

POST /api/v1/mllm/train

```
{ "source_job_id": "class_job_xyz789",   // classification job to take
embeddings from
  "base_model": "distilgpt2",             //
distilgpt2|llama3.2-1b|tinyllama|bert-multilingual
  "fusion_method": "concat",
  "fine_tune_method": "lora",             // full | lora | qlora
  "lora_rank": 8,
  "quantization": "none",                 // none | 4bit | 8bit
  "learning_rate": 2e-5,
  "epochs": 10,
  "batch_size": 16,
  "task": "classification"                // classification | contrastive |
both
}
```

Response (202): { job_id, status_url, websocket_url }

GET /api/v1/mllm/status/{job_id}

```
{ "job_id": "mllm_abc", "status": "running",
  "epoch": 6, "total_epochs": 10,
  "train_loss": 0.34, "val_loss": 0.41,
  "train_accuracy": 0.81, "val_accuracy": 0.76,
  "step": "fine_tuning_lora_layer_6" }
```

GET /api/v1/mllm/export/{job_id}?format=pytorch|gguf|onnx

Download trained Geo-MLLM weights. pytorch = .pt file, gguf = for llama.cpp deployment, onnx = for cross-platform inference.

GET /api/v1/mllm/model-card/{job_id}

Returns auto-generated model card as JSON: architecture, dataset, metrics, fusion config, hyperparameters. Frontend renders as downloadable PDF.

5.5 Digital Twin — NL Geo-Query (NOW MVP)

POST /api/v1/query

```
{ "question": "show residential areas near industrial zones",
  "grid_id": "grid_abc123",          // REQUIRED
  "language": "auto",                // auto | ar | en
  "mllm_job_id": "mllm_abc"          // optional — use trained Geo-MLLM
}
```

```
Response: {
  "cell_ids": [12, 45, 78, 102],
  "explanation": "Areas classified as Residential within 500m of Industrial
cells",
  "explanation_ar": "مناطق سكنية على بعد 500م من المناطق الصناعية",
  "confidence": 0.88
}
```

5.6 Training Lab (Classification)

POST /api/v1/train

Start classifier fine-tuning job. Multipart: labels file + config JSON string.

Response: { job_id, status_url }

GET /api/v1/train-status/{job_id}

```
{ "epoch": 12, "loss": 0.34, "accuracy": 0.81, "status": "running",
  "val_loss": 0.38, "val_accuracy": 0.79 }
```

5.7 Other Endpoints (Retained from v2.0)

Endpoint	Method	Purpose
/api/v1/area-status/{job_id}	GET	Poll loading progress
/api/v1/classify-status/{job_id}	GET	Poll classification progress
/api/v1/export/{job_id}?format=x	GET	Download classification result
/api/v1/thumbnails/{grid_id}/{cell_id}.jpg	GET	Satellite image patch
/api/v1/jobs/{job_id}	DELETE	Cancel any running job

5.8 WebSocket Events

Phase	WebSocket URL	Key Events
Loading	ws://.../ws/progress/{load_job_id}	step, progress, completed, failed
Classification	ws://.../ws/progress/{classify_job_id}	step, progress, batch_complete (with partial GeoJSON)
MLLM Training	ws://.../ws/progress/{mllm_job_id}	epoch, train_loss, val_loss, train_acc, val_acc
Classifier Training	ws://.../ws/progress/{train_job_id}	epoch, loss, accuracy, completed

6. Mock Data Mode — Full Specification

All endpoints including new v3.0 endpoints (graph-topology, evaluate, mllm/train, query) have MSW mock handlers. Mock mode enabled by VITE_MOCK_MODE=true.

Endpoint	Delay	Mock Behaviour
POST /load-area	0ms	Returns job_id; polling simulates 3s load in steps
GET /grid/{id}/preview	200ms	Returns mock GeoJSON 144-cell Cairo grid
GET /grid/{id}/graph-topology	300ms	Returns mock road graph: ~620 nodes, ~890 edges for Cairo bbox
POST /classify	0ms	Returns job_id; WS simulates 12 batches of 12 cells over 8s
GET /classification-result/{id}	300ms	Full mock GeoJSON 144 cells, 4 modalities, GNN or MLP results
POST /evaluate	500ms	Returns mock metrics: Acc=0.81, MacroF1=0.78, SpatialAcc=0.74
POST /mllm/train	0ms	Returns job_id; WS simulates 10 epochs with realistic loss curve over 15s
GET /mllm/export/{id}	1000ms	Returns placeholder .pt file download
GET /mllm/model-card/{id}	400ms	Returns auto-generated mock model card JSON
POST /query	600ms	Returns pre-defined cell sets for 5 demo questions (AR + EN)
POST /train	0ms	Returns job_id; simulates 15-epoch training with WS events
GET /export/{id}	500ms	Returns mock GeoJSON blob download
DELETE /jobs/{id}	0ms	Returns 200 OK

Mock NL Query Scenarios

The 5 pre-defined mock query responses cover the demo use cases:

Query	Language	Mock response
Show residential areas near commercial zones	EN	Returns 22 cells, explanation in English
اين المناطق التجارية في القاهرة؟	AR	Returns 31 cells, explanation in Arabic + English
Find industrial zones near highways	EN	Returns 8 cells with road density > 5.5
Show areas with high road density	EN	Returns 19 cells, road_density > 5.0
المناطق السكنية ذات الكثافة المنخفضة	AR	Returns 14 low-confidence residential cells

7. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	Fast build, HMR, TypeScript
Mapping	MapLibre GL JS	GeoJSON layers, graph topology, NL query highlights
UI	Mantine v7	Rich components, RTL support (Arabic), dark mode
State	Zustand	Minimal boilerplate, URL persistence
Data fetching	TanStack Query v5	Async state, caching, retry
HTTP	Axios	Interceptors, auth headers
WebSocket	Native WS API	Real-time progress; mock fallback hook
Mock API	MSW (Mock Service Worker)	Intercepts at SW level — zero API client changes
Charts	Recharts	MLLM loss/accuracy curves, evaluation metrics, histogram
PDF export	jsPDF	Model card generation on client side
Testing	Vitest + RTL + Playwright	Unit + E2E covering all 16 acceptance criteria
i18n	react-i18next	AR/EN string keys, RTL layout class on html element

8. Component Architecture

8.1 Pages / Routes

Route	Page & Components
/	Main Dashboard — MapView, Sidebar, CellDetailPanel, StatusBar
/mllm-builder	MLLM Builder — ModelSelector, FusionConfig, HyperparamForm, TrainingChart, LogPanel, ExportPanel
/digital-twin	Digital Twin — NLQueryInput, QueryHistory, MapView (query highlights), SpatialExplanationPanel
/training-lab	Training Lab — LabelUpload, ClassifierConfig, TrainingChart, MetricsPanel
/evaluation	Evaluation Panel — MetricCards, ConfusionMatrix, GroundTruthUpload, F1BarChart
/ablation	Ablation Comparison — MultiRunSelector, SideBySideMaps, MetricsComparisonTable

8.2 Zustand Store

```
interface DashboardStore {
  bbox: [number,number,number,number] | null;
  gridSize: 200 | 500 | 1000;
  modalities: ('poi'|'image'|'graph'|'text')[];  // 4 options now
  modelType: 'mlp' | 'gnn';                       // NEW
  fusionMethod: 'concat'|'weighted'|'attention';
  loadJobId: string | null;
  classifyJobId: string | null;
  gridId: string | null;
  classificationResult: GeoJSON.FeatureCollection | null;
  evaluationResult: EvaluationResult | null;         // NEW
  mllmJobId: string | null;                          // NEW
  mllmModelReady: boolean;                           // NEW
  nlQueryHistory: QueryHistoryItem[];                // NEW
  pinnedCells: CellFeature[];
  selectedCell: CellFeature | null;
  layers: {                                          // NEW: graphTopology
added
    result:boolean; poi:boolean; roads:boolean;
    graphTopology:boolean; satellite:boolean; queryHighlight:boolean;
  };
  status: 'idle'|'loading'|'classifying'|'evaluating'|'done'|'error';
  progress: number;
  progressStep: string;
}
```

9. Data Flow — Complete Pipeline

9.1 Phase 2-4: Data Collection & Graph

1. User draws bbox → cell count estimate shown.
2. POST /load-area with 4 modalities → WebSocket loading progress.
3. GET /grid/{id}/preview → empty cells render on map.
4. GET /grid/{id}/graph-topology → road nodes + edges render if GNN selected.

9.2 Phase 6-7: Classification

5. User sets modalities (4 checkboxes) + model type (GNN or MLP).
6. POST /classify with model_type + modalities → WebSocket batch_complete events.
7. Progressive cell reveal with confidence opacity. Distribution histogram updates.
8. GET /classification-result → full GeoJSON stored in store.

9.3 Phase 10: Evaluation

9. Evaluation tab opens automatically after classification.
10. Without ground truth: distribution + confidence stats shown.
11. User uploads ground truth CSV → POST /evaluate → metrics panel populates.
12. Accuracy, F1 per class, Spatial Accuracy, confusion matrix displayed.

9.4 Phase 8-9: MLLM Builder

13. User navigates to /mllm-builder.
14. Selects base model + fusion method + LoRA config.
15. POST /mllm/train → WebSocket emits epoch-level loss/accuracy.
16. Live training chart updates. Training log shows real-time output.
17. On completion: model available for NL query and for download.

9.5 Phase 11: Digital Twin

18. User navigates to /digital-twin.
19. Types query in Arabic or English (or clicks example chip).
20. POST /query with { question, grid_id, mllm_job_id (optional) }.
21. Response highlights matching cells in cyan on map.
22. Spatial explanation shown in panel with Arabic translation if applicable.

10. Acceptance Criteria

#	Criterion	Test Method
1	User draws bbox, sees cell count estimate, loads area with 4 modalities	Playwright
2	Graph topology layer renders road nodes and edges when GNN selected	Visual QA + Playwright
3	Classify with GNN or MLP — cells appear progressively with confidence opacity	Playwright
4	Text modality checkbox is independent from POI — ablation (Text-only) works	Playwright
5	Cell click shows class, %, POI, thumbnail, road density, graph embedding norm	Playwright
6	Evaluation panel shows Accuracy, F1, Spatial Accuracy after ground truth upload	Playwright + Vitest
7	Confusion matrix renders correctly for 3-class output	Visual QA
8	MLLM Builder: select base model + config + start training + live loss chart	Playwright
9	MLLM training completes; model can be downloaded in pytorch format	Playwright
10	NL query in English returns highlighted cells + English explanation	Playwright
11	NL query in Arabic returns highlighted cells + Arabic + English explanation	Playwright
12	Ablation mode runs 2 combinations and shows side-by-side maps	Manual
13	Export: GeoJSON, evaluation CSV, MLLM model weights all download correctly	Playwright
14	Cancel job button works for load, classify, MLLM train, and classifier train	Playwright
15	Mock Mode: all 14 above criteria pass with VITE_MOCK_MODE=true	Full Playwright suite
16	No horizontal scroll at 1280x720; Arabic RTL query renders correctly	Browser DevTools

11. Implementation Guide for AI-Assisted Build

Use this guide with Claude Code, Cursor, or any AI coding agent. Each phase is self-contained and testable in Mock Mode before the next phase begins. Implement in order — each phase depends on the previous.

Phase 1 — Project Scaffold & Mock Layer (Day 1)

23. Vite + React 18 + TypeScript. Install all dependencies from Section 7.
24. MSW init: npx msw init public/. Create handlers.ts with all 14 mock endpoints.
25. Zustand store matching the full interface in Section 8.2.
26. react-i18next setup with ar.json and en.json string files. Set dir=rtl when Arabic.
27. Route setup: /, /mllm-builder, /digital-twin, /training-lab, /evaluation, /ablation.

Phase 2 — Main Dashboard Map (Day 2)

28. MapLibre GL map with satellite tiles. Bbox draw tool. Keyboard shortcut hook.
29. Sidebar: AreaSection (search + draw + cell estimate), GridSection, 4-checkbox ModalitySection, ModelTypeSelector (GNN/MLP), ActionSection, LayerSection.
30. Wire POST /load-area → WebSocket → grid preview layer.
31. Wire GET /graph-topology → GraphTopologyLayer (nodes as circles, edges as lines, toggle G key).

Phase 3 — Classification & Progressive Reveal (Day 3)

32. POST /classify with model_type + modalities → batch_complete WebSocket events.
33. MapLibre fill-opacity paint expression: [interpolate, linear, confidence, 0.51, 0.35, 1.0, 0.95].
34. CellDetailPanel: all fields including graph_embedding_norm + text_embedding_norm.
35. ComparisonPanel for pinned cells. DistributionHistogram in StatusBar.

Phase 4 — Evaluation Panel (Day 4)

36. EvaluationPanel route /evaluation. MetricCards: Accuracy, Macro F1, Weighted F1, Spatial Accuracy.
37. GroundTruthUpload: drag-and-drop CSV. Calls POST /evaluate. Populates metrics.
38. ConfusionMatrix: 3x3 table with colour intensity mapped to cell count.
39. F1BarChart: horizontal Recharts bar chart, one bar per class per modality combination.

Phase 5 — MLLM Builder (Day 5-6)

40. Route /mllm-builder. BaseLLMSelector dropdown, FusionMethodSelector.
41. HyperparamForm: lr, epochs, batch_size, LoRA rank, quantization, fine_tune_method.

42. TrainingChart: Recharts dual-line chart (train_loss + val_loss, train_acc + val_acc) per epoch.
43. LogPanel: scrolling pre-formatted text updated by WebSocket epoch events.
44. ExportPanel: Download .pt / .gguf / .onnx buttons. Model card PDF generation via jsPDF.

Phase 6 — Digital Twin NL Query (Day 6)

45. Route /digital-twin. NLQueryInput with RTL auto-detect and Arabic keyboard support.
46. ExampleChips: 5 clickable queries (2 Arabic, 3 English) from mock scenarios.
47. POST /query → add cyan #00B4D8 fill layer to map for cell_ids in response.
48. SpatialExplanationPanel: show explanation + explanation_ar side-by-side.
49. QueryHistory: last 10 queries stored in Zustand, one-click replay.

Phase 7 — Ablation + Training Lab + Polish (Day 7)

50. AblationPage /ablation: MultiRunSelector, SideBySideMapGrid, MetricsComparisonTable.
51. TrainingLabPage /training-lab: LabelUpload, ClassifierConfig, TrainingChart.
52. Full Playwright suite: all 16 acceptance criteria in mock mode.
53. Switch VITE_MOCK_MODE=false, point to live FastAPI at localhost:8000, verify parity.

12. Risks & Mitigations

Risk	Mitigation
GNN inference slow on CPU	Show estimated time warning; recommend GPU worker in Docker
Graph topology layer crowded (many edges)	Max 500 nodes rendered; cluster smaller nodes; reduce opacity at low zoom
Arabic RTL layout breaks	Use Mantine dir prop; test all panels with arabic text of various lengths
MLLM training OOM on RTX 2070	Default to LoRA + 4-bit quantization; add memory estimate in UI before start
GGUF export format unsupported on backend	Offer pytorch only as guaranteed format; gguf/onnx as beta exports
MSW does not support WebSocket natively	Use setInterval-based mock progress hook; same interface as real WS hook
Large area timeout (500+ cells)	Block submit if estimate > 500; show amber warning > 300

Risk	Mitigation
WebSocket disconnect during MLLM training	Auto-reconnect; on failure switch to polling /mllm/status every 2s
Spatial Accuracy metric undefined in sklearn	Implement as custom metric: % cells where class matches majority of 8 neighbours

13. Academic Alignment Checklist

This section confirms that every phase and deliverable of the student project document is now covered by the PRD v3.0.

	Student Doc Phase	Status in PRD v3.0	Covered by
✓	Phase 2: Road / POI / Satellite data	Fully covered	FR-03, Section 5.1
✓	Phase 3: Data cleaning (WGS84)	Backend only — correct	NFR note
✓	Phase 4: Graph G=(V,E)	Fully covered	FR-08, GET graph-topology
✓	Phase 6: Fusion (4 modalities)	Fully covered	FR-09 to FR-12
✓	Phase 7: GNN + CNN + Text Encoder	Fully covered	FR-13 to FR-16
✓	Phase 8: Custom MLLM build	Fully covered	FR-34 to FR-45, Section 5.4
✓	Phase 9: Training (CL + Contrastive)	Fully covered	FR-53 to FR-56
✓	Phase 10: Evaluation (Acc, F1, Spatial)	Fully covered	FR-26 to FR-31, Section 5.3
✓	Phase 11: Digital Twin + NL query	Fully covered — now MVP	FR-46 to FR-52, Section 5.5
✓	Phase 12: Geo-MLLM export + demo	Fully covered	FR-44, FR-45, FR-52
~	Distillation / Quantization / LoRA	Quantization + LoRA in MLLM Builder (FR-37,38). Distillation is model-level, not UI.	FR-37, FR-38

Qualification score after v3.0 gap resolution

All 5 critical and warning gaps from the v2.0 qualification review are now fully resolved. The PRD is qualified for build against the student project objectives. Estimated qualification score: 9.5/10. The

remaining 0.5 is knowledge distillation which is a model-level operation outside the scope of any UI PRD.

14. Future Roadmap (Post-MVP)

Release	Feature	Notes
v3.1	3D Digital Twin	MapLibre 3D extrusion of classified cells, height = confidence score
v3.1	Mobile layout	Responsive for tablets (1024px min) — Mantine breakpoints
v3.2	Knowledge distillation UI	Upload teacher + student model configs; monitor distillation training
v3.2	User sessions + auth	Save/restore analyses; JWT auth; per-user model storage
v4.0	External data overlay	Population density, official land-use zoning layers
v4.0	Multi-city comparison	Run same pipeline on multiple cities, compare Digital Twins

Document v3.0 | April 2026 | AI Engineering Team | All gaps resolved